Node.js — Day 10

The path Module: Joining and Reading Paths

Learning goal: Understand Node.js's built-in **path** module and use it to safely build paths and extract filenames.

> **Connection:** Day 6 introduced built-in modules; Day 10 applies that idea practically with the **path** module.

1. What is the path Module?

The **path** module is a built-in Node.js module used for working with file and directory paths.

```
const path = require("path");
```

Because it is built into Node.js, it does not need to be installed with npm.

2. Why Use path Instead of String Concatenation?

A backend often needs to construct paths such as uploads, documents, or generated files. Instead of manually joining strings, use path methods.

```
const filePath = "uploads/" + "profile.jpg";
```

Manual string joining can be error-prone across operating systems because path separators can differ.

```
const path = require("path");
const filePath = path.join("uploads", "profile.jpg");
```

path.join() handles the path separator appropriately for the operating system.

3. path.join()

Purpose: combines path pieces into one properly formed path.

```
const path = require("path");

const filePath = path.join("uploads", "profile.jpg");

console.log(filePath);
```

The result represents a path such as **uploads/profile.jpg** on systems that use `/` as the separator.

> **Important:** path.join() creates a path string. It does not create the actual file or read the file.

4. path.basename()

Purpose: returns the last part of a path, usually the filename.

```
const path = require("path");

const filePath = "/users/bhavana/notes.txt";

console.log(path.basename(filePath));
```

Output:

```
notes.txt
```

It works with the path string. It does not read the file or check whether the file exists.

5. Practical Backend Example

Imagine a backend receives or stores a file under this path:

```
uploads/documents/resume.pdf
```

You can extract the filename with:

```
const path = require("path");

const filePath = path.join("uploads", "documents", "resume.pdf");

console.log(path.basename(filePath));
```

Result:

```
resume.pdf
```

This pattern is useful when handling uploaded files and other file-related backend operations.

6. path Module Mental Model

```
path.join()
    ↓
COMBINE PATH PIECES

path.basename()
    ↓
GET LAST PART / FILENAME
```

Both methods operate on **path strings**; they do not themselves perform file I/O.

7. Important Rules

• **path** is a built-in Node.js module.

• Load it with **require("path")** in the CommonJS style we are learning.

• **path.join()** combines path parts safely for the operating system.

• **path.basename()** gets the final part of a path, usually the filename.

• Neither method creates, reads, deletes, or checks the actual file.

• Use path utilities instead of manually assuming path separators.

8. Day 10 Quick Recap

path → built-in module for working with paths.

path.join() → combines path parts into a proper path.

path.basename() → gets the last part / filename from a path.

These methods work with **path strings**, not the contents of the actual files.

Practice Completed

Day 10: **3 / 3 questions correct** ■

Key Takeaway

The path module helps Node.js build and inspect file paths safely. path.join() constructs a path; path.basename() extracts the filename.